LAB # 05

Customer Churn Prediction using ANN

Lab Task:-

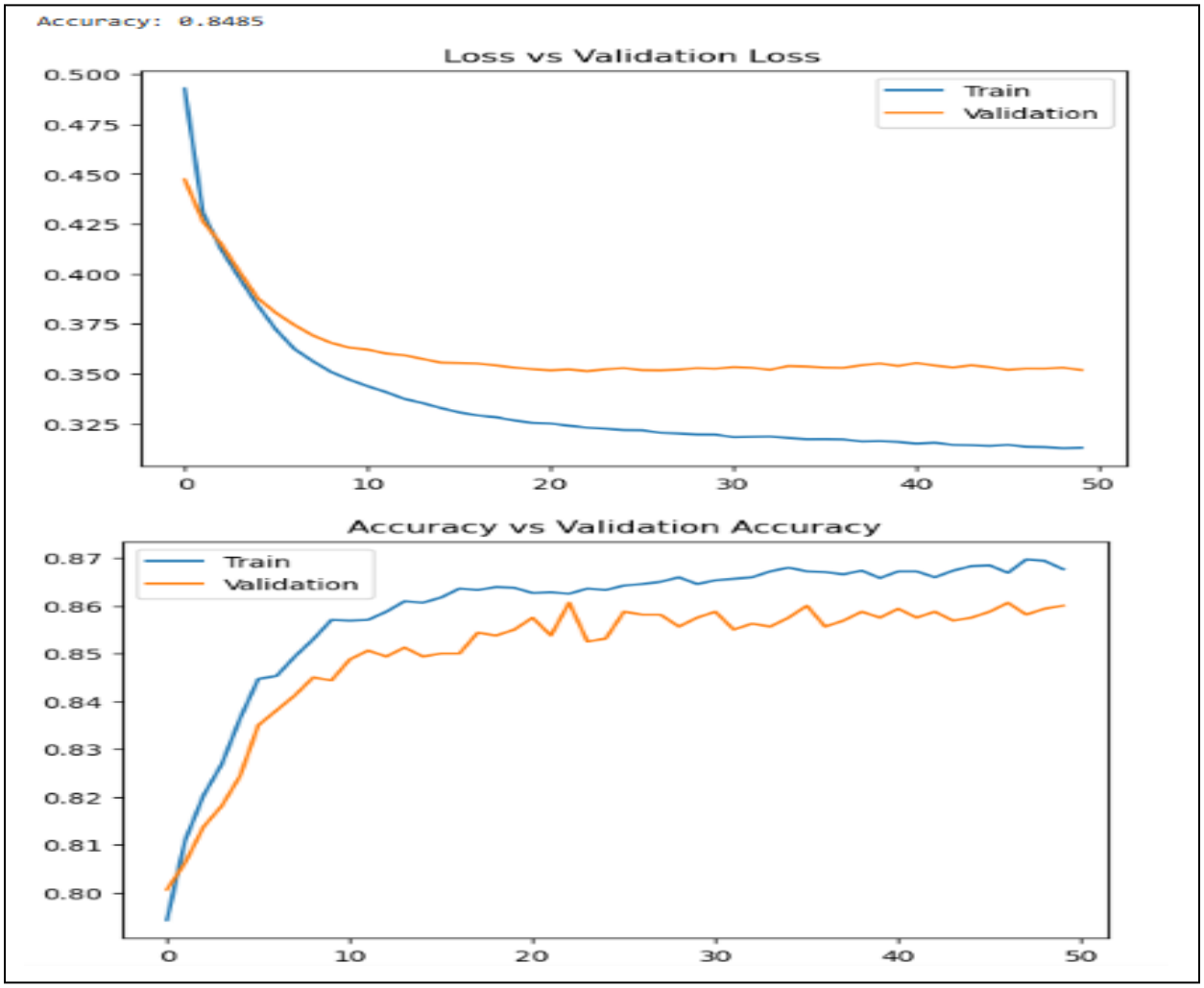
- Download a suitable binary classification dataset from Kaggle. Load and preprocess the dataset by handling missing values, encoding categorical variables, and scaling the features. Build a Multilayer Perceptron (MLP) model using an Ann. Train the model on the dataset to predict the target variable. Evaluate the model's performance using the accuracy score and test the trained model on new, unseen data samples to assess its prediction capability.

Code:-

```
import pandas as pd
import numpy as np

df = pd.read_csv("Churn_Modelling.csv")

df.drop(columns=['RowNumber','CustomerId','Surname'], inplace=True)

df = pd.get_dummies(df, columns=['Geography','Gender'], drop_first=True)

X = df.drop(columns=['Exited'])
y = df['Exited']

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()
model.add(Dense(16, activation='relu', input_dim=X_train.shape[1]))
model.add(Dense(16, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

history = model.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2)

y_pred = model.predict(X_test)
y_pred = (y_pred > 0.5).astype(int)

from sklearn.metrics import accuracy_score
print("Accuracy:", accuracy_score(y_test, y_pred))

import matplotlib.pyplot as plt

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title("Loss vs Validation Loss")
plt.legend(["Train", "Validation"])
plt.show()

plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title("Accuracy vs Validation Accuracy")
plt.legend(["Train", "Validation"])
plt.show()
```

Output:-

```
Epoch 1/50
C:\Users\LAPCOM\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input_shap
ayer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.7942 - loss: 0.4928 - val_accuracy: 0.8006 - val_loss: 0.4473
Epoch 2/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8109 - loss: 0.4306 - val_accuracy: 0.8062 - val_loss: 0.4263
Epoch 3/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8203 - loss: 0.4122 - val_accuracy: 0.8138 - val_loss: 0.4150
Epoch 4/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8269 - loss: 0.3979 - val_accuracy: 0.8181 - val_loss: 0.4011
Epoch 5/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8364 - loss: 0.3842 - val_accuracy: 0.8244 - val_loss: 0.3878
Epoch 6/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8447 - loss: 0.3720 - val_accuracy: 0.8350 - val_loss: 0.3804
Epoch 7/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8453 - loss: 0.3624 - val_accuracy: 0.8381 - val_loss: 0.3744
Epoch 8/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8494 - loss: 0.3563 - val_accuracy: 0.8413 - val_loss: 0.3692
Epoch 9/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8530 - loss: 0.3509 - val_accuracy: 0.8450 - val_loss: 0.3655
Epoch 10/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8570 - loss: 0.3471 - val_accuracy: 0.8444 - val_loss: 0.3631
Epoch 11/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8569 - loss: 0.3438 - val_accuracy: 0.8487 - val_loss: 0.3621
Epoch 12/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8570 - loss: 0.3409 - val_accuracy: 0.8506 - val_loss: 0.3602
Epoch 13/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8587 - loss: 0.3375 - val_accuracy: 0.8494 - val_loss: 0.3593
Epoch 14/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8609 - loss: 0.3354 - val_accuracy: 0.8512 - val_loss: 0.3575
Epoch 15/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8606 - loss: 0.3329 - val_accuracy: 0.8494 - val_loss: 0.3556
Epoch 16/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8617 - loss: 0.3307 - val_accuracy: 0.8500 - val_loss: 0.3554
Epoch 17/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8636 - loss: 0.3292 - val_accuracy: 0.8500 - val_loss: 0.3551
Epoch 18/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8633 - loss: 0.3283 - val_accuracy: 0.8544 - val_loss: 0.3542
Epoch 19/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8639 - loss: 0.3267 - val_accuracy: 0.8537 - val_loss: 0.3531
Epoch 20/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8637 - loss: 0.3254 - val_accuracy: 0.8550 - val_loss: 0.3523
Epoch 21/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8627 - loss: 0.3251 - val_accuracy: 0.8575 - val_loss: 0.3517
Epoch 22/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8628 - loss: 0.3240 - val_accuracy: 0.8537 - val_loss: 0.3522
Epoch 23/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8625 - loss: 0.3230 - val_accuracy: 0.8606 - val_loss: 0.3513
Epoch 24/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8636 - loss: 0.3225 - val_accuracy: 0.8525 - val_loss: 0.3523
Epoch 25/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8633 - loss: 0.3218 - val_accuracy: 0.8531 - val_loss: 0.3528
Epoch 26/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8642 - loss: 0.3217 - val_accuracy: 0.8587 - val_loss: 0.3518
Epoch 27/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8645 - loss: 0.3205 - val_accuracy: 0.8581 - val_loss: 0.3517
Epoch 28/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8650 - loss: 0.3201 - val_accuracy: 0.8581 - val_loss: 0.3521
Epoch 29/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8659 - loss: 0.3196 - val_accuracy: 0.8556 - val_loss: 0.3528
Epoch 30/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8645 - loss: 0.3196 - val_accuracy: 0.8575 - val_loss: 0.3525
Epoch 31/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8653 - loss: 0.3183 - val_accuracy: 0.8587 - val_loss: 0.3533
Epoch 32/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8656 - loss: 0.3185 - val_accuracy: 0.8550 - val_loss: 0.3530
Epoch 33/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8659 - loss: 0.3186 - val_accuracy: 0.8562 - val_loss: 0.3520
Epoch 34/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8672 - loss: 0.3179 - val_accuracy: 0.8556 - val_loss: 0.3539
Epoch 35/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8680 - loss: 0.3172 - val_accuracy: 0.8575 - val_loss: 0.3536
Epoch 36/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8672 - loss: 0.3173 - val_accuracy: 0.8600 - val_loss: 0.3531
Epoch 37/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8670 - loss: 0.3172 - val_accuracy: 0.8556 - val_loss: 0.3530
Epoch 38/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8666 - loss: 0.3161 - val_accuracy: 0.8569 - val_loss: 0.3543
Epoch 39/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8673 - loss: 0.3163 - val_accuracy: 0.8587 - val_loss: 0.3552
Epoch 40/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8658 - loss: 0.3159 - val_accuracy: 0.8575 - val_loss: 0.3539
Epoch 41/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8672 - loss: 0.3150 - val_accuracy: 0.8594 - val_loss: 0.3554
Epoch 42/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8672 - loss: 0.3155 - val_accuracy: 0.8575 - val_loss: 0.3541
Epoch 43/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8659 - loss: 0.3144 - val_accuracy: 0.8587 - val_loss: 0.3531
Epoch 44/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8673 - loss: 0.3143 - val_accuracy: 0.8569 - val_loss: 0.3543
Epoch 45/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8683 - loss: 0.3139 - val_accuracy: 0.8575 - val_loss: 0.3533
Epoch 46/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8684 - loss: 0.3145 - val_accuracy: 0.8587 - val_loss: 0.3520
Epoch 47/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8669 - loss: 0.3135 - val_accuracy: 0.8606 - val_loss: 0.3526
Epoch 48/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8697 - loss: 0.3133 - val_accuracy: 0.8581 - val_loss: 0.3526
Epoch 49/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8694 - loss: 0.3127 - val_accuracy: 0.8594 - val_loss: 0.3531
Epoch 50/50
200/200 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8677 - loss: 0.3130 - val_accuracy: 0.8600 - val_loss: 0.3519
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```


Accuracy: 0.8485
Loss vs Validation Loss
Train
Validation
0.500
0.475
0.450
0.425
0.400
0.375
0.350
0.325
0
10
20
30
40
50
Accuracy vs Validation Accuracy
Train
Validation
0.87
0.86
0.85
0.84
0.83
0.82
0.81
0.80
0
10
20
30
40
50